

Лекция-методичка по обучению с подкреплением

*Темы: единый подход; иерархическое обучение с подкреплением;
определение суб-компонент; алгоритмы MAXQ и ALISP;
мульти-агентное обучение с подкреплением; методы полной и не прямой координации*

Учебный материал с примерами, сравнительными таблицами и практическими замечаниями

1. Зачем нужен единый подход к сложным задачам обучения с подкреплением

В классическом обучении с подкреплением агент взаимодействует со средой, наблюдает состояния, выбирает действия и получает вознаграждения. Однако при переходе к реальным задачам быстро возникает несколько проблем: пространство состояний слишком велико, полезные решения состоят из длинных последовательностей действий, в среде могут действовать несколько агентов одновременно, а достижение общей цели часто требует координации на разных уровнях абстракции.

Единый подход — это идея рассматривать сложную задачу не как монолит, а как систему взаимосвязанных уровней управления. На нижнем уровне остаются элементарные действия, а на более высоких уровнях появляются подзадачи, субцели, макро-действия, правила координации и механизмы обмена информацией. В результате обучение становится более структурированным: агент не просто ищет отдельное действие, а учится решать подзадачи, комбинировать их и передавать результаты на верхний уровень принятия решений.

В современной литературе такой подход объединяет несколько направлений: иерархическое обучение с подкреплением, обучение на основе опций и макро-действий, декомпозицию задачи, мульти-агентное обучение с подкреплением, централизованное и децентрализованное планирование, а также методы координации и коммуникации. Несмотря на различия, их связывает общая идея: сложность можно победить, если разложить задачу на управляемые компоненты и правильно организовать обмен знаниями между ними.

2. Единый подход: общая схема

Единый подход к сложным RL-системам можно описать следующей цепочкой:

- описать исходную глобальную цель системы;
- выделить уровни принятия решений: стратегический, тактический, исполнительный;
- определить суб-компоненты или подзадачи, которые можно обучать отдельно;
- задать интерфейсы между компонентами: входы, выходы, локальные цели, сигналы завершения;
- выбрать способ координации: централизованный, полностью координированный, не прямой, через среду, через совместную награду, через ограниченные сообщения;
- согласовать обучение локальных компонентов с обучением общей политики.

Такой подход одинаково полезен как в задаче одного сложного агента, так и в задаче многих агентов. В первом случае подкомпоненты — это иерархия навыков одного агента. Во втором — это отдельные агенты, которые вместе должны реализовать поведение системы.

Уровень	Что решается	Типичные объекты	Пример
Стратегический	Выбор долгосрочной цели	подзадачи, режимы, цели	доставить груз во все зоны
Тактический	Определение способа достижения подцели	маршруты, роли, распределение задач	назначить робота на сектор
Исполнительный	Элементарные действия	движения, повороты, захват	сделать шаг вперед
Координационный	Согласование нескольких процессов	протоколы обмена, правила уступки	избежать столкновения
Обучающий	Обновление моделей и ценностей	Q-функции, модели мира, критики	уточнить ценность опции

Таблица показывает, что единый подход не заменяет классический RL, а встраивает его в многослойную структуру, где решения на одном уровне становятся контекстом для другого.

3. Иерархическое обучение с подкреплением

Иерархическое обучение с подкреплением (Hierarchical Reinforcement Learning, HRL) изучает способы представления задачи в виде иерархии целей и подзадач. Вместо того чтобы на каждом шаге выбирать только одно примитивное действие, агент может выбирать более крупную сущность — например, «дойти до двери», «взять ключ», «исследовать сектор», «доставить объект в точку выгрузки».

Главное преимущество HRL состоит в сокращении эффективной глубины поиска. Если агент учится напрямую на уровне примитивных действий, ему приходится просчитывать длинные цепочки. При наличии подзадач значительная часть этой работы переносится на нижние уровни и переиспользуется в разных ситуациях. Навык «обойти препятствие» или «подойти к станции зарядки» может использоваться многократно в составе разных глобальных планов.

В HRL важны три идеи: временная абстракция, структурная декомпозиция и повторное использование навыков. Временная абстракция означает, что выбранная высокоуровневая команда может выполняться много шагов подряд. Структурная декомпозиция означает, что задача разбивается на части. Повторное использование навыков означает, что одна и та же подзадача может решаться во многих сценариях и потому ее выгодно учить отдельно.

3.1. Основные единицы HRL

- Примитивное действие — одно элементарное действие среды.
- Опция или макро-действие — временно протяженная политика с условием запуска и условием завершения.
- Подзадача — локальная задача со своей целью, локальной функцией ценности или локальным вознаграждением.

- Иерархия — дерево или граф зависимостей, связывающий верхние и нижние уровни управления.
- Псевдонаграда — искусственный сигнал, который помогает обучать подзадачу, даже если глобальная награда редка.

3.2. Пример иерархии

Рассмотрим мобильного робота на складе. Глобальная цель — обработать заказ. Возможная иерархия:

- уровень 1: обработать заказ;
- уровень 2: найти товар, подъехать к стеллажу, взять товар, доставить к упаковке;
- уровень 3: построить путь до сектора, объехать препятствия, выровняться у стеллажа;
- уровень 4: шаг вперед, поворот влево, поворот вправо, захват.

Такая структура особенно полезна, когда один и тот же путь, маневр или захват встречается во многих заказах. Вместо повторного обучения каждого случая с нуля система переиспользует уже освоенные суб-компоненты.

4. Определение суб-компонент

Ключевой этап проектирования HRL-системы — выделение суб-компонент. Если разложение удачно, обучение ускоряется, перенос между задачами растёт, интерпретируемость улучшается. Если разложение неудачно, система становится громоздкой, а подзадачи оказываются бесполезными или конфликтующими.

Суб-компонента — это фрагмент поведения, который имеет собственный смысл, собственный входной контекст и собственный критерий завершения. Это может быть локальная цель, навык, подполитика, подмодель мира, роль агента или даже шаблон координации.

4.1. Признаки хорошей суб-компоненты

- Локальная завершённость: подзадача имеет понятный признак успеха.
- Повторяемость: навык нужен во многих эпизодах или в разных задачах.
- Ограниченный контекст: решение подзадачи зависит от небольшой части состояния.
- Слабая связанность: изменение внутренней реализации компоненты мало влияет на остальные.
- Возможность локального вознаграждения: компоненту можно обучать, не дожидаясь глобального финала.

4.2. Способы выделения подзадач

- Экспертная декомпозиция: структуру задаёт человек на основе понимания предметной области.
- Выделение по субцелям: подзадачи строятся вокруг достижимых промежуточных состояний.

- Выделение по узким местам: подзадачи соответствуют трудным участкам траектории.
- Выделение по пространственной структуре: разные комнаты, зоны, секторы, этапы маршрута.
- Выделение по ролям: разведчик, перевозчик, охранник, координирующий агент.
- Автоматическое обнаружение опций: алгоритм сам находит полезные временные абстракции.

Подход	Плюсы	Риски
Экспертный	понятная структура, управляемость	возможна субъективность и жесткость
По субцелям	хорошо согласуется с наградами	сложно выбрать действительно полезные субцели
По графу состояний	можно автоматизировать	нужен надежный анализ траекторий
По ролям	естественно для мульти-агентных задач	роли могут быть слишком жесткими
Автоматическое обнаружение	может найти неожиданные навыки	полученные опции не всегда интерпретируемы

На практике часто используют смешанный подход: базовая иерархия задается экспертом, а внутри нее отдельные опции или локальные политики дообучаются автоматически.

5. Алгоритм MAXQ

MAXQ — один из классических формализмов иерархического обучения с подкреплением. Он описывает задачу как декомпозицию MDP на подзадачи. Каждая подзадача может быть либо примитивным действием, либо составной задачей, которая вызывает другие подзадачи.

Главная идея MAXQ — разложить функцию ценности не только по состояниям и действиям, но и по структуре иерархии. Вместо одной общей Q-функции используются локальные функции, связанные с тем, какой подкомпонент выбран и сколько ценности он добавляет к общей цели.

5.1. Интуиция MAXQ

Если глобальная задача «обслужить клиента» включает подзадачи «подойти к клиенту», «выяснить заказ», «доставить товар», то ценность глобального выбора можно представить как сумму локальной ценности текущей подзадачи и ценности дальнейшего продолжения. Таким образом, MAXQ учится не одной гигантской функции, а набору более компактных функций, связанных с иерархией.

5.2. Компоненты MAXQ-декомпозиции

- Иерархия задач: корневая задача и ее подзадачи.
- Политика выбора дочерней подзадачи внутри составной задачи.

- Функция завершения: когда считается, что подзадача выполнена.
- Функция локальной ценности и функция продолжения (completion function).
- Возможность рекурсивного вызова подзадач и повторного использования навыков.

5.3. Что дает MAXQ

- уменьшение размерности локальных задач;
- возможность интерпретировать поведение через дерево подзадач;
- естественное повторное использование одинаковых навыков;
- ускорение обучения в задачах со структурой;
- упрощение переноса знаний при изменении верхнего уровня цели.

5.4. Ограничения MAXQ

- нужно спроектировать разумную иерархию;
- слишком жесткая декомпозиция может ограничить оптимальность;
- локальные цели могут конфликтовать с глобальной целью;
- при неправильной абстракции агент учится хорошо решать подзадачи, но плохо решать исходную задачу.

6. Алгоритм ALISP

ALISP (Adaptive, state-abstracted, Lisp-like programming for RL) — это подход, в котором поведение агента задается высокоуровневой программной схемой. В отличие от полностью «плоского» обучения, здесь заранее фиксируется частичная структура решения: последовательности шагов, циклы, условия выбора, точки вызова подзадач. Алгоритм не перебирает все возможные стратегии на равных, а учится внутри пространства программ, совместимого с заданной схемой.

Идея ALISP особенно полезна там, где предметная область уже частично понятна. Человек может задать каркас: например, сначала доберись до нужной зоны, затем обследуй объекты, после этого вернись к базе. Агенту остается научиться конкретизировать внутренние решения: какой маршрут выбрать, когда завершить поиск, как реагировать на частичную наблюдаемость и неопределенность.

6.1. Особенности ALISP

- использование процедурного каркаса поведения;
- возможность задавать подзадачи как фрагменты программы;
- абстрагирование состояния внутри отдельных подпрограмм;
- комбинация экспертных знаний и обучения;
- снижение пространства поиска за счет ограничений на допустимые стратегии.

6.2. Когда ALISP особенно полезен

- если известен общий сценарий решения, но неясны оптимальные локальные действия;

- если нужно встроить RL в уже существующую бизнес-логику или процедурный регламент;
- если важна интерпретируемость и возможность вручную проверять структуру поведения;
- если требуется безопасное ограничение пространства допустимых стратегий.

6.3. Сравнение MAXQ и ALISP

Критерий	MAXQ	ALISP	Комментарий
Основная идея	декомпозиция задачи на подзадачи	обучение внутри программного каркаса	оба подхода структурируют поиск
Источник структуры	дерево/граф задач	частично заданная программа	ALISP сильнее опирается на процедурную форму
Гибкость	высокая при удачной иерархии	ограничена каркасом, но безопаснее	компромисс между свободой и контролем
Интерпретируемость	хорошая	очень высокая	ALISP близок к читаемой логике
Риск	неудачная иерархия	слишком жесткие ограничения	в обоих случаях важен дизайн структуры

7. Мульти-агентное обучение с подкреплением

В мульти-агентном обучении с подкреплением (Multi-Agent Reinforcement Learning, MARL) несколько агентов одновременно обучаются и действуют в общей среде. Каждый агент имеет собственные наблюдения, действия и, возможно, собственную награду. При этом среда становится нестационарной с точки зрения отдельного агента: пока один агент учится, политики остальных тоже изменяются.

Именно поэтому MARL значительно сложнее одиночного RL. Нужно учитывать совместную динамику, проблему частичной наблюдаемости, конкуренцию за ресурсы, необходимость согласования ролей, опасность паразитного поведения и феномен кредитного назначения: если награда общая, как понять вклад конкретного агента?

7.1. Где возникает MARL

- робототехнические группы: несколько роботов перевозят грузы;
- транспортные системы: автомобили координируют движение на перекрестках;
- игровые среды: команда ботов взаимодействует против соперника;
- распределенные вычисления: сервисы согласуют использование ресурсов;
- энергосистемы и логистика: множество локальных контроллеров решают общую задачу.

7.2. Основные трудности MARL

- нестационарность среды для каждого агента;
- взрыв размерности совместного пространства действий;
- сложность назначения заслуги отдельным агентам;
- частичная наблюдаемость и локальные знания;
- потребность в координации и устойчивости к ошибкам партнеров.

8. Методы полной координации

Методы полной координации предполагают, что агенты могут совместно формировать решение, ориентируясь на общую информацию или общий критерий качества. В предельном случае система действует как единый «суперагент», у которого просто много исполнительных каналов.

Полная координация особенно полезна, когда цена ошибки велика, действия агентов сильно взаимозависимы, а обмен информацией технически возможен. Например, при совместной перевозке тяжелого груза, управлении энергосетью или координации группы дронов в узком пространстве.

8.1. Формы полной координации

- Централизованная политика: единый контроллер выдает команды всем агентам.
- Централизованный критик при децентрализованном исполнении: обучение использует глобальную информацию, но на этапе применения агенты действуют локально.
- Совместная Q-функция: оценивается ценность совместного действия команды.
- Планирование с общим буфером знаний: агенты пишут и читают общую память.
- Распределение ролей центральным менеджером: верхний уровень задает, кто и что делает.

8.2. Преимущества полной координации

- высокое качество согласованных решений;
- меньше конфликтов между агентами;
- проще реализовать общую стратегию и глобальные ограничения;
- лучше решается проблема кредитного назначения при наличии глобального критика.

8.3. Недостатки полной координации

- высокие требования к связи и синхронизации;
- плохая масштабируемость при росте числа агентов;
- риск единой точки отказа;
- часто требуется доступ к глобальному состоянию, которого в реальной системе нет.

9. Методы непрямой координации

Непрямая координация означает, что агенты не обязаны непрерывно обмениваться полными планами или подчиняться единому центральному управляющему. Вместо этого координация возникает через структуру награды, через правила среды, через локальные сигналы, через роли, маркеры, ожидания, соглашения и следы в среде.

Классический пример — стигмергия: один агент меняет среду, а другой использует это изменение как косвенный сигнал. Муравьи оставляют феромоны, роботы могут пометать уже обследованные зоны, сервисы — резервировать ресурсы в общей таблице, а беспилотники — использовать виртуальные коридоры и приоритеты.

9.1. Формы непрямой координации

- Координация через общую награду или штрафы за конфликты.
- Координация через правила доступа к ресурсам.
- Координация через следы в среде: пометки, феромоны, флаги, резервирования.
- Координация через обученные роли и специализацию.
- Координация через локальные протоколы уступки и приоритета.
- Координация через прогноз поведения других агентов.

9.2. Плюсы непрямой координации

- лучшая масштабируемость;
- меньшая нагрузка на связь;
- устойчивость к отказу отдельных узлов;
- естественность для распределенных систем и частичной наблюдаемости.

9.3. Ограничения непрямой координации

- агенты могут долго учиться не конфликтовать;
- поведение может быть менее оптимальным, чем при полной координации;
- сложно добиться строгих гарантий качества;
- косвенные сигналы могут быть неоднозначны или запаздывать.

10. Подробные примеры

10.1. Пример 1. Складские роботы

Предположим, на складе работает команда из четырех мобильных роботов. Нужно собирать заказы, не сталкиваясь, не создавая пробок и не разряжая батареи в критические моменты. Единый подход строится так:

- на верхнем уровне система распределяет заказы по роботам;
- на среднем уровне каждый робот разбивает свою работу на подзадачи: доехать до зоны, взять товар, отвезти в упаковку, вернуться;
- на нижнем уровне выполняются примитивные действия движения и маневров;

- координация может быть полной для распределения заказов и не прямой для локального движения в коридорах.

MAXQ здесь естественно задает дерево задач одного робота, а мульти-агентный уровень определяет распределение ролей между роботами. Непрямая координация может выражаться в резервировании участков маршрута или в приоритетах движения на перекрестках.

10.2. Пример 2. Команда дронов-разведчиков

Есть группа дронов, обследующих территорию после чрезвычайной ситуации. Глобальная цель — как можно быстрее покрыть карту и обнаружить критические объекты. Возможная иерархия:

- корень: обследовать всю территорию;
- подзадачи: назначить сектора, долететь до сектора, исследовать сектор, передать отчет, вернуться на базу;
- локальные навыки: удержание высоты, обход зоны риска, стабилизация на ветру.

Полная координация полезна при первичном делении карты и при распределении редких ресурсов связи. Непрямая координация возникает, когда дроны помечают уже просмотренные участки и избегают дублирования. ALISP-подход особенно удобен, если есть регламент миссии: старт, разлет по секторам, локальное обследование, периодическая передача результатов, возврат.

10.3. Пример 3. Интеллектуальный трафик

На сети перекрестков каждый светофор рассматривается как агент. Общая цель — минимизировать суммарное время ожидания и количество заторов. Полная координация возможна через центральный диспетчер, а не прямая — через локальный обмен данными о плотности потока и через общие ограничения на фазы. Иерархия может строиться так: стратегический уровень выбирает режим района, локальный уровень управляет конкретными фазами светофора.

11. Практические рекомендации по проектированию

- Не дробите задачу слишком мелко: чрезмерная иерархия затрудняет обучение и сопровождение.
- Проверяйте, действительно ли подзадача переиспользуется. Если нет, иерархия может быть лишней.
- Разделяйте, что должно быть централизованным, а что — локальным. Стратегия сверху, реакция снизу — часто удачная схема.
- Продумывайте сигналы завершения подзадач: неясный критерий завершения ломает всю иерархию.
- Для мульти-агентных систем отдельно проектируйте механизм назначения заслуги и вознаграждения.
- Сначала делайте простую интерпретируемую иерархию, затем усложняйте ее автоматическим обучением опций.

12. Краткий словарь терминов

Термин	Смысл
Временная абстракция	Представление длительных фрагментов поведения как единого действия или подзадачи.
Подзадача	Локальная задача внутри общей иерархии.
Опция	Макро-действие с условием запуска, внутренней политикой и условием завершения.
Псевдонаграда	Дополнительная награда для обучения подзадачи.
Полная координация	Сильное согласование действий агентов на основе общей информации или общего управляющего.
Непрямая координация	Согласование через среду, общие правила, награды, роли или косвенные сигналы.
MAXQ	Формализм декомпозиции MDP на иерархию задач и локальных функций ценности.
ALISP	Подход к RL с частично заданной программной структурой поведения.

13. Выводы

Единый подход позволяет рассматривать сложные RL-системы как совокупность уровней, подзадач и координационных механизмов. Это особенно важно в реальных задачах, где плоское обучение по примитивным действиям оказывается слишком дорогим или неинтерпретируемым.

Иерархическое обучение с подкреплением делает возможным повторное использование навыков, уменьшает глубину поиска и приближает обучение к естественной структуре сложной деятельности. MAXQ подчеркивает декомпозицию функции ценности по иерархии задач, а ALISP — использование программного каркаса, который ограничивает и направляет поиск.

В мульти-агентном случае главным дополнением становится координация. Полная координация дает высокое качество, но плохо масштабируется. Непрямая координация более гибкая и естественная для распределенных систем, однако обычно требует более тонкой настройки сигналов и механизмов взаимодействия. На практике наиболее эффективными часто оказываются гибридные решения: централизованное обучение и локальное исполнение, глобальное распределение ролей и локальная адаптация, иерархия целей плюс непрямая координация на исполнительном уровне.

14. Контрольные вопросы

- Почему для сложных задач RL полезен единый подход, основанный на декомпозиции?
- Чем иерархическое обучение отличается от плоского RL?
- Какими свойствами должна обладать хорошая суб-компонента?
- В чем состоит основная идея MAXQ?
- В каких случаях ALISP может быть предпочтительнее более свободных методов?
- Почему мульти-агентное обучение приводит к нестационарности среды?
- Чем методы полной координации отличаются от непрямой координации?
- Почему в реальных системах часто используют гибридные схемы координации?